

SOC Automation Pipeline — MVP

Platform: Cortex XSOAR 8 (cloud) · **Status:** end-to-end functional in UKDEV with Microsoft Sentinel as the first source

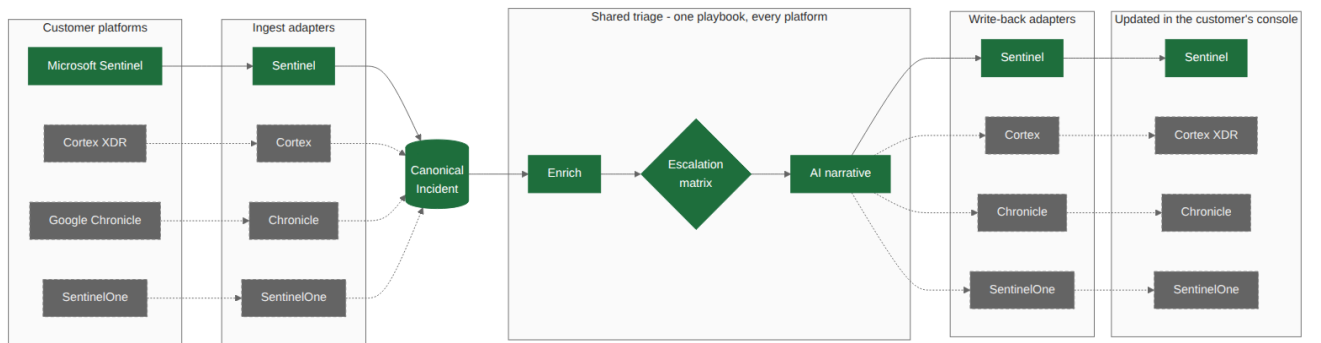
1. Summary

One triage pipeline for every customer and every SIEM.

Incidents from any source are converted into a single canonical format at the edge, then flow through one shared triage process: enrichment, a deterministic escalation matrix, an AI-written narrative, and write-back to the customer's own console. Adding a new platform means writing a small adapter at each end. The triage logic in the middle never changes.

Today: Sentinel is fully wired, front to back. **Next:** Cortex XDR and Google Chronicle plug into the same pipeline — the routing for them already exists.

2. One pipeline, many platforms



Solid = built. Dashed = planned — the shared playbook already has a branch for each and sends them to manual review until the adapter exists.

The adapter contract

Each platform needs exactly two small sub-playbooks. Everything else is shared.

Adapter	Job	Contract
Ingest	Pull whatever that platform can give — entities, related alerts, rule logic, severity — and normalise it	Must output a <code>CanonicalIncident</code> object and an <code>ExtractedEntities</code> list
Sync	Push the SOC's decision back into that platform	Accepts a <code>Comment</code> and a <code>Disposition</code> ; writes the comment, mirrors the status

Between those two points the shared playbook references only the canonical object. There is no Sentinel field, no Cortex field, no vendor-specific context anywhere in the triage logic — so it cannot drift per platform.

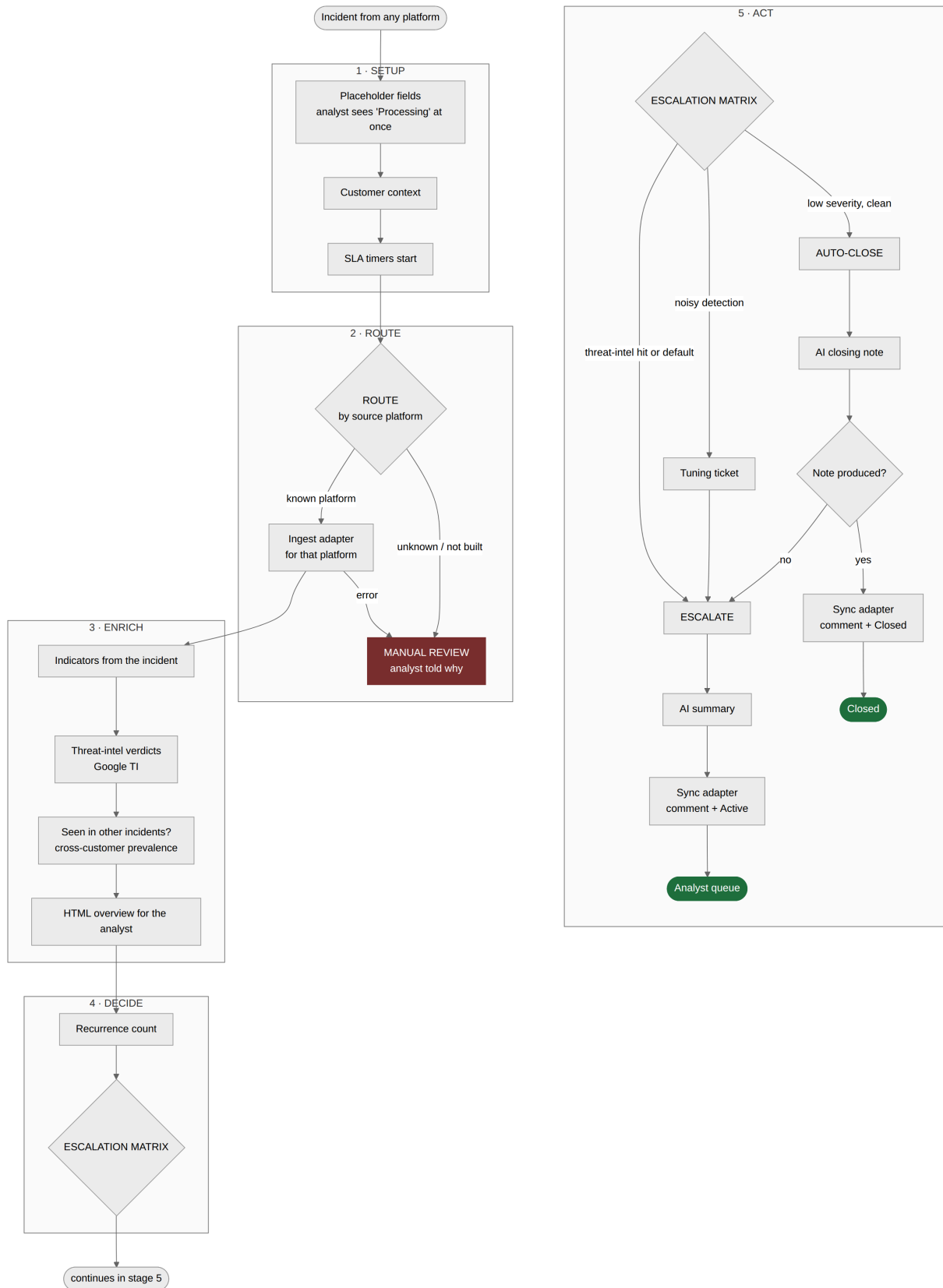
What adding a platform takes

1. Configure the platform's XSOAR integration for the customer
2. Write the ingest adapter (normalise to canonical — the Sentinel one is the template)
3. Write the sync adapter (comment + status)
4. Point the existing routing branch at them

No change to enrichment, the matrix, AI prompts, escalation or auto-close.

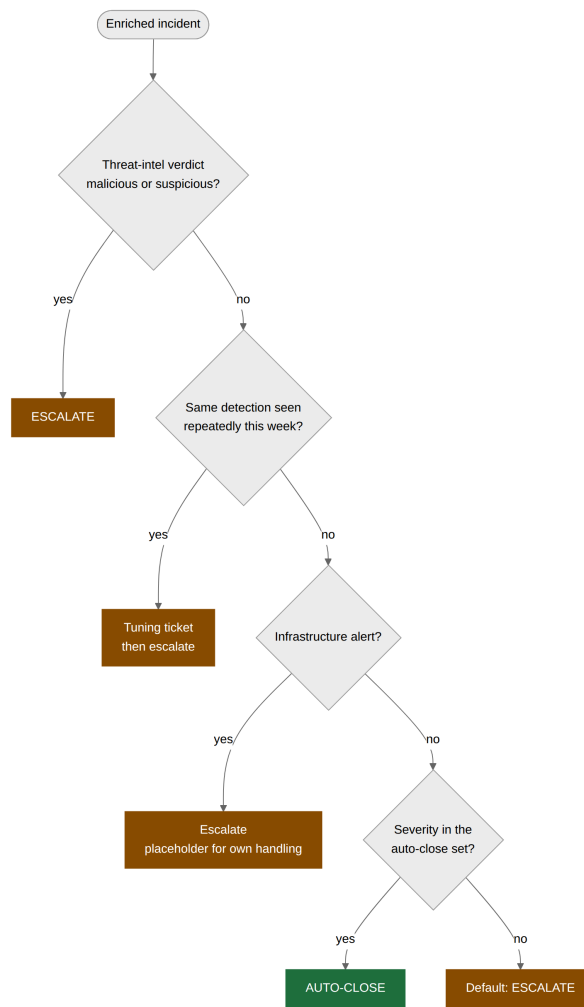
3. The shared triage flow

This is what every incident goes through regardless of source. Five stages, top to bottom.



The escalation matrix

Rules run in order; first match wins. The default is escalate, so nothing is dropped or closed by accident.



Auto-close is the only path that removes work from the queue. It is reached only after the other rules have not matched, and it will not close unless the AI produced a usable closing note. If that fails, the incident escalates.

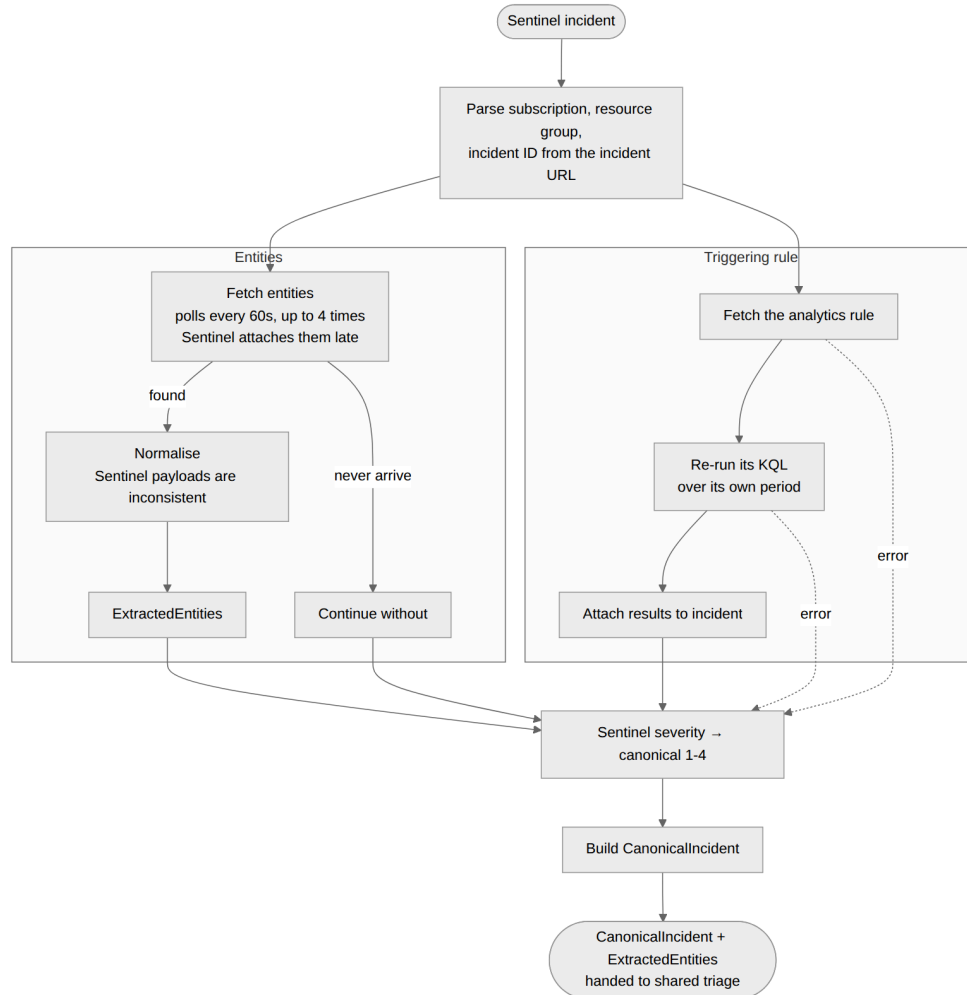
Thresholds, the auto-close severity set, and the recurrence window are all playbook inputs — tunable per customer without touching the logic.

4. Worked example: Microsoft Sentinel

The same pipeline, followed through for the one platform that is live.

4.1 Sentinel ingest adapter

Everything Sentinel-specific lives here and only here.

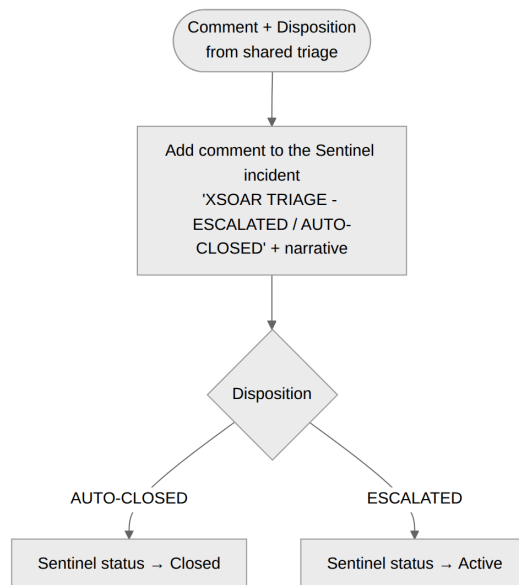


Two Sentinel quirks handled here so the shared layer never sees them: entities arrive asynchronously (hence the polling loop), and entity payloads change shape between API calls (hence the normaliser).

4.2 Shared triage

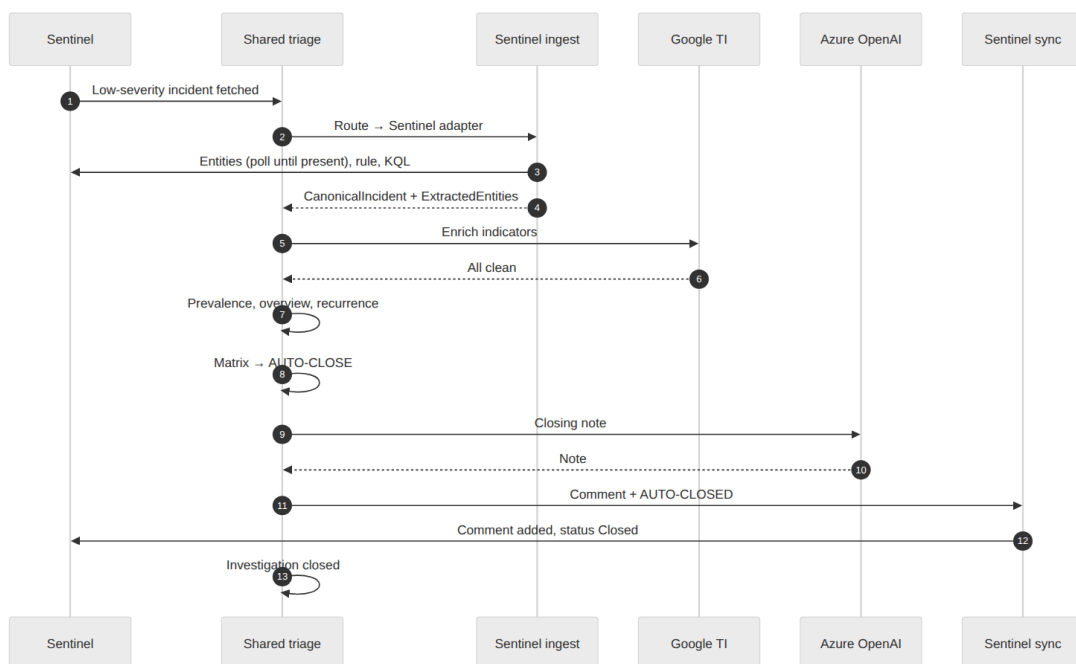
Runs exactly as in section 3. Nothing in it knows the incident came from Sentinel.

4.3 Sentinel sync adapter



The customer's Sentinel console shows what the SOC concluded and why, without anyone opening XSOAR.

4.4 One incident, end to end



Swap `Sentinel ingest` and `Sentinel sync` for the Cortex equivalents and the diagram is otherwise identical.

5. Built to degrade, not to stop

Every non-fatal failure ends with an analyst looking at the incident. The only fatal failure is being unable to build the canonical object, and that goes to manual review with the fields updated to say so.

If this fails...	The pipeline...
Unknown platform or adapter not built	Sends to manual review, tells the analyst why
Entities never arrive from the source	Continues without them; matrix escalates by default
Threat-intel unavailable	Uses whatever verdicts exist; no auto-close on a false sense of safety
AI summary fails	Writes a fallback narrative; analyst and customer still get a note
AI closing note fails or is empty	Escalates instead of closing
Write-back to the platform fails	Logs it; the triage outcome stands

6. Roadmap

Designed and held back from the MVP only because they need XSOAR configuration that doesn't exist yet. Each is a small, isolated addition to the shared playbook — none change the adapter model.

Item	Value	Needs
Cortex XDR and Chronicle adapters	Second and third platforms on the same pipeline	Two sub-playbooks each
Per-customer config	Thresholds and instances keyed on customer, one playbook serves all	One JSON list
Observability flags	Every route an incident took, on the incident — dashboards for free	Two incident fields
Fingerprint dedup	Detect duplicates on detection + entities, not name; link and close automatically	One searchable field
Customer allow-lists	Skip enrichment on known-benign values — fewer TI calls, less noise	One list per customer

Suggested order: per-customer config, then the second platform. The second platform is the proof that the model holds.

7. Metrics from day one

- Auto-close rate and reopen rate per customer — the safety check on auto-close
- Time from fetch to disposition
- AI failure rate by prompt type
- Tuning tickets raised per detection rule
- Write-back failure rate